İZMİR EKONOMİ ÜNİVERSİTESİ

Design Document for “Sports Manager”

Team Trinity

Kaan Bilgili – 20210602012

Karya Tanrıkulu – 20230602064

Fatma Nur Erdoğan - 20220602030

Contents

1. **Introduction and Problem Definition** 3
   - 1.1 Project Objectives 3
   - 1.2 System Functionality 3
2. **Requirements** 3
   - 2.1 Functional Requirements 3
   - 2.2 Non-Functional Requirements 3
3. **System Design and Architecture** 4
   - 3.1 Layered Architecture 4
     - 3.1.1 UI Layer 5
     - 3.1.2 Application Layer 5
     - 3.1.3 Domain Layer 5
     - 3.1.4 Game Rules Layer 5
   - 3.2 Match Simulation Flow 6
   - 3.3 GameFacade Design 6
4. **Domain Model** 7
   - 4.1 Overview 7
   - 4.2 Core Entities 7
   - 4.3 Relationships 8
5. **Sport Plugin Architecture** 9
   - 5.1 Sport Interface 9
   - 5.2 Example Sport Implementation 10
6. **GUI Design** 10
   - 6.1 Overview 10
   - 6.2 Menu Screen 10
   - 6.3 Sport Selection Screen 11
   - 6.4 League Dashboard Screen 12
   - 6.5 Team Management Screen 12
   - 6.6 Match Simulation Screen 13
   - 6.7 GUI Navigation Flow 14
7. **Risks and Assumptions** 15
   - 7.1 Risks 15
   - 7.2 Assumptions 15
8. **Conclusion** 15

1.Introduction and Problem Definition

1.1 Project Objectives

The aim of the Sports Manager project is to develop a graphical user interface (GUI) based sports management simulation game where the user can manage a team and compete in a league. Unlike other manager games that focus on only one sport, this project is designed to support multiple sports by using object-oriented programming principles. Rules and behaviours specific to each sport are separated from the user interface so that new sports can be added to the system more easily in the future.

1.2 System Functionality

In this game, teams, players, and match schedules are generated automatically. The user can make decisions such as selecting the lineup, changing tactics, and making substitutions during matches. During the season the user can follow match results, league standings, and player conditions.

2. Requirements

2.1 Functional Requirements

FR1: The system shall allow the user to start a new game.
FR2: The system shall allow the user to select a sport to manage.
FR3: The system shall generate teams and players automatically.
FR4: The system shall generate league schedules for the season.
FR5: The user shall be able to view the match schedule.
FR6: The user shall be able to view league rankings.
FR7: The user shall be able to select the starting lineup before matches.
FR8: The user Shall be able to change tactics.
FR9: The user shall be able to make substitutions during match simulations.
FR10: The system shall simulate matches automatically.
FR11: The system shall update league standings after each match.
FR12: The system shall handle player injuries.

2.2 Non-Functional Requirements

NFR1: The system must support adding new sports without modifying the GUI.
NFR2: The application must run on Windows OS.
NFR3: The user interface must be responsive.
NFR4: The system should be modular and expandable.

3. System Design and Architecture

3.1 Layered Architecture

Overview

The Sports Manager system is designed using a layered architecture. Layered architecture is a software design pattern that organizes an application into horizontal layers, each with specific roles.

Each layer is assigned a specific responsibility. Layers communicate only with the layer directly below them, ensuring a clear separation between system components. This makes the system easier to understand and supports adding new features in the future.

Another advantage is the changes in one layer have minimal impact on the other layers. This reduces the complexity and makes the application easier to maintain.

The main layers of the Sports Manager system are described below:

- UI Layer
- Application Layer
- Domain Layer
- Game Rules Layer

These layers work together to ensure that the user interface, system logic, and sport rules remain separated and organized.

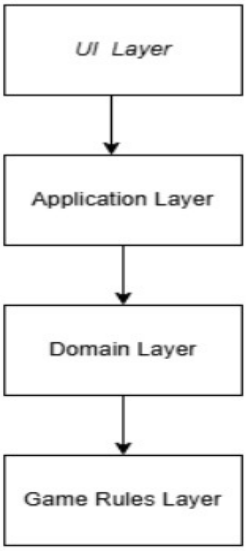

Figure 3.1 : Layered Architecture

3.1.1 UI Layer

UI stands for User Interface. The UI layer is responsible for handling all interactions between the user and the system. It provides graphical components such as menus, buttons, tables, and screens that allow the user to interact with the application. Through this layer, the user can start a new game, view league standings, and access other game features. The UI layer is responsible for displaying information and receiving input from the user, while the application logic is handled in another layer of the system.

3.1.2 Application Layer

The Application Layer controls the principal operations of the system and acts as a bridge between the user interface and the application. This layer receives the requests from the UI layer and processes them by controlling the operations of the system. This layer is in charge of controlling the general flow of the game, such as starting a new game, creating teams, creating schedules, and starting match simulations. The Application Layer handles the main system operations and links the user interface to the system logic.

3.1.3 Domain Layer

The Domain Layer includes the main models, rules, and logic. This layer is independent of the UI layer. This layer primarily works with the main entities of the system. These include teams, players, matches, and leagues. This layer includes the relationships between entities. Teams are composed of players, a league includes several teams, and a match is played between two teams. One of the advantages of the domain layer is that the key logic of the system is kept independent of the user interface.

3.1.4 Game Rules Layer

The Game Rules Layer is the one that defines how the game is played with regards to the specific sport that the application is handling. As the application may be handling different games with their respective rules and mechanisms, this is the part that deals with the management of the same. It is the one that defines how the matches are simulated, how the scoring is done, and how the teams interact with each other.

3.2 Match Simulation Flow

The match simulation process refers to the process that the system uses in the generation of the outcome of the match between two teams. The system does not generate the outcome directly, but it uses a series of steps in the generation of the outcome of how the match is progressing.

First, the system selects the match that it should trigger for the simulation, and it is chosen from the fixture list. The application layer then prepares the required data, where it retrieves the teams that are involved in the match.

Then, the game rules are used in the simulation of the match events. The game rules are used in the generation of the outcome of the match.

After the simulation process, the outcome of the match is generated, and then the standing of the league is updated based on the outcome of the match. The system uses the structure in the generation of the outcome of the match in an orderly manner.

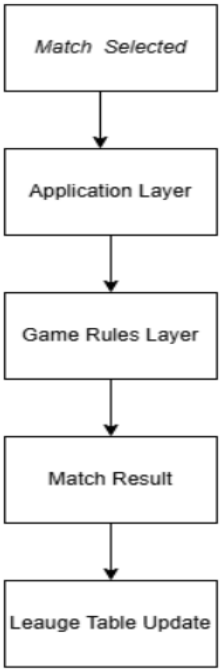

Figure 3.2 : Match Simulation Flow

3.3 GameFacade Design

The GameFacade can be considered a coordination point that simplifies the interaction between the user interface and the internal components of the system. Instead of providing direct access to several domain classes for the user interface, it provides access to several operations through a single access point.

From the point of view of the user interface, the system can be considered to be providing several operations to the user. From the point of view of the internal structure, it can be considered to be providing access to classes such as Season, League, Match, or Fixture.

4. Domain Model

4.1 Overview

The domain model represents the core entities required to simulate a sports management environment in the project. This model focuses on modeling the core elements of league-based competition such as seasons, leagues, teams and matches.These entities describe how the competition is structured and how teams interact with one another during the season.

In the system, a season represents the general competition period during which matches are played. Each season contains one or more leagues with multiple teams competing. Teams consist of players who participate in matches and help the team perform. Matches are simulated events between two teams that produce results that directly affect the league rankings.

The domain layer is designed to remain separate from specific rules of the sport and graphical user interface components. This allows the application's core structure to be reused when new sports are added and supports the project's goal of maintainability.

4.2 Core Entities

Season: Refers to the period during which all matches are played. It manages the progress of the leagues and ensures that the competition schedule is followed correctly.
League: Refers to an organized competition where multiple teams compete against each other. It is responsible for creating match schedules and updating league rankings for the whole season.
Team: Represents a group of players competing together in matches. It stores plate information and tracks team performance throughout the competition.
Player: Represents an individual competitor with specific qualities and fitness status. Players can be selected for matches and may experience injuries or changes in performance.
Match: Represents a simulated event between two teams in the league. It generates a result that affects team rankings and overall competition progress.
StandingEntry: Refers to a team's ranking data in the league table. It tracks points, match results and the team's relative position compared to other teams.
Fixture: Represents the schedule of matches that will be played during the season. It organizes matches in a structured order and makes sure that teams compete according to the planned timeline.

4.3 Relationships

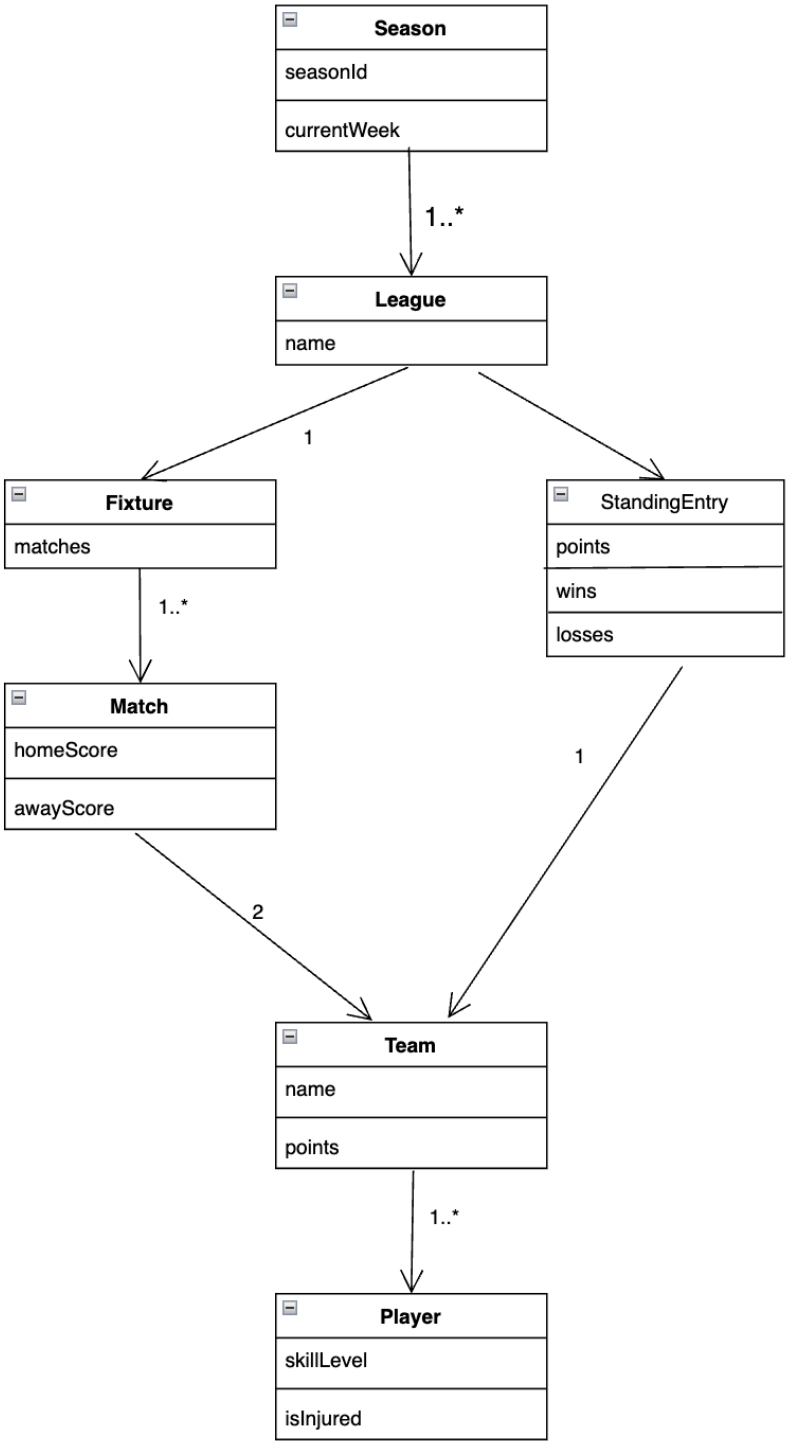

Figure 4.1 : UML Diagram

5. Sport Plugin Architecture

In this system , the rules that guide a particular sport are not hardcoded within the main structure of the application. Instead, these rules are set up separately to allow for the use of different rules for different sports.

The rules for a particular sport may be different from those for another. These differences are what make the system keep the rules for a particular sport in a separate structure. This way, the main structure of the application will not be affected by the rules for a particular sport.

In this case, the main parts of the application will not be affected by the rules for a particular sport.

5.1 Sport Interface

In the Sports Manager system, various sports may have their own rules and match structure.Due to these differences, it is not possible to have a specific implementation that is applicable to all sports. However, a common interface is used to define the basic behaviors that must be supported by all sports.

The Sport Interface specifies a set of operations that allow the system to interact with a sport. These operations may include various operations such as simulating a match, using a scoring system, and structuring a team. Although each sport will implement these operations differently, it is clear that all will be structured in a similar manner to the Sport Interface.

By using this approach, it is not necessary for the main system to be aware of the internal implementation of each sport. It will only be able to interact with it through the interface. This makes it easier to add new sports to the system in the future because a new sport can be added easily without affecting any other part of the system.

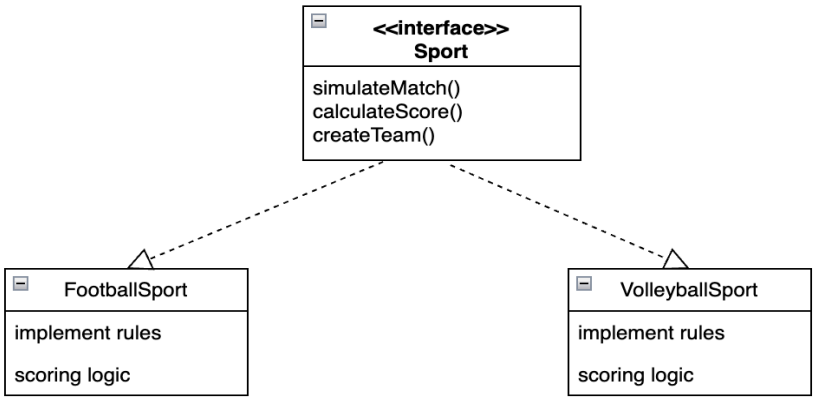

Figure 5.1 : Sport Abstraction Interface Diagram

5.2 Example Sport Implementation

To better understand the application of the interface in real life, a sport can be added to the system by defining a class based on the Sport Interface. This class includes the specific rules related to the chosen sport.

In the case of the football sport, the implementation includes the definition of the number of players in a team, the definition of the goals, and the definition of the progress of the match over time. These rules are included in the implementation of the sport instead of the general system.

The application uses the interface to interact with the sport without referencing the implementation of the classes. Therefore, the system can support the addition of more sports later in the development by adding a new implementation based on the same interface definition.

6. GUI Design

6.1 Overview

The graphical user interface (GUI) provides a clean, structured, and intuitive interaction environment that supports the overall gameplay experience. It enables users to navigate between different stages of the game such as starting a new season, selecting the sport, managing the team lineup, viewing league standings, and simulating matches.

The interface is designed to remain independent from sport-specific rules by interacting with the application layer through defined abstractions. This design ensures that new sports can be integrated into the system without requiring major changes to the graphical structure. Consistent layouts, clearly visible action buttons, and structured data presentation using tables and panels are used throughout the interface to improve usability, readability, and gameplay continuity.

6.2 Menu Screen

The menu screen is the first interface displayed when the application starts and provides the user with the main entry options of the system. It includes two primary action buttons: the New Game button, which initiates the game flow and directs the user to the sport selection screen, and the Exit button, which allows the user to close the application. The screen follows a centered layout design to present the available actions clearly and improve usability.

Figure 6.1 : Menu Screen

6.3 Sport Selection Screen

The sport selection screen is displayed after the user starts a new game and allows the user to choose which sport to manage. Available sports are presented as selectable buttons arranged in a grid layout to ensure clarity and scalability. The sport buttons enable the user to select preferred sport, such as Football or Volleyball, while the Start Game button confirms the selection and proceeds to the league dashboard screen. This interface structure supports extensibility by allowing new sports to be added without requiring changes to the overall layout.

Figure 6.2 : Sport Selection Screen

6.4 League Dashboard Screen

The league dashboard screen serves as the central hub of the application during the season. It enables the user to monitor league standings and navigate between different game modules. A sidebar navigation menu is used to provide access to core features such as viewing the league table, checking the match fixture, managing the team, and starting match simulations. Additionally, the Next Match button allows the user to initiate the upcoming scheduled match directly from this screen. League standings are presented in a structured table format to ensure clear visualization of statistical information and team rankings.

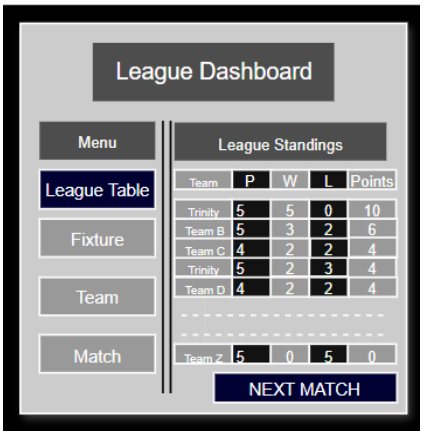

Figure 6.3 : League Dashboard Screen

6.5 Team Management Screen

The team management screen allows the user to organize and prepare the squad before matches. On this screen, the user can view the full list of available players and select the starting lineup based on player attributes and tactical needs. Action buttons such as Add Player, Change Tactics, and Confirm Lineup support squad configuration and strategic decision-making. Player information is presented using structured tables and lists to improve readability and facilitate efficient team selection.

Figure 6.4 : Team Management Screen

6.6 Match Simulation Screen

The match simulation screen displays the progress of a match in real time through score updates and event logs. Users can follow key match events such as goals, substitutions, or injuries as they occur. Interaction during simulation is limited to maintain realism, but the user can still influence gameplay through action buttons such as Continue, Change Tactic, and Substitute.

When the match ends, the system generates the final result and updates league standings accordingly. The user is then redirected back to the league dashboard, allowing continuous gameplay and smooth transition between preparation and competition phases.

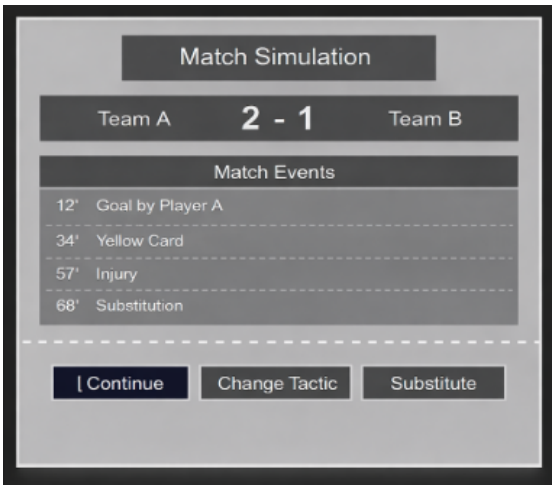

Figure 6.5 : Match Simulation Screen

6.7 GUI Navigation Flow

The diagram above illustrates the navigation flow between the main screens of the application. The user starts from the menu screen and proceeds to the sport selection screen when a new game is initiated. After selecting the sport, the system directs the user to the league dashboard, which serves as the central control point of the game. From the dashboard, the user can access the team management screen to configure the lineup or start match simulations. Once a match is completed, the system returns the user to the league dashboard, ensuring continuous gameplay progression.

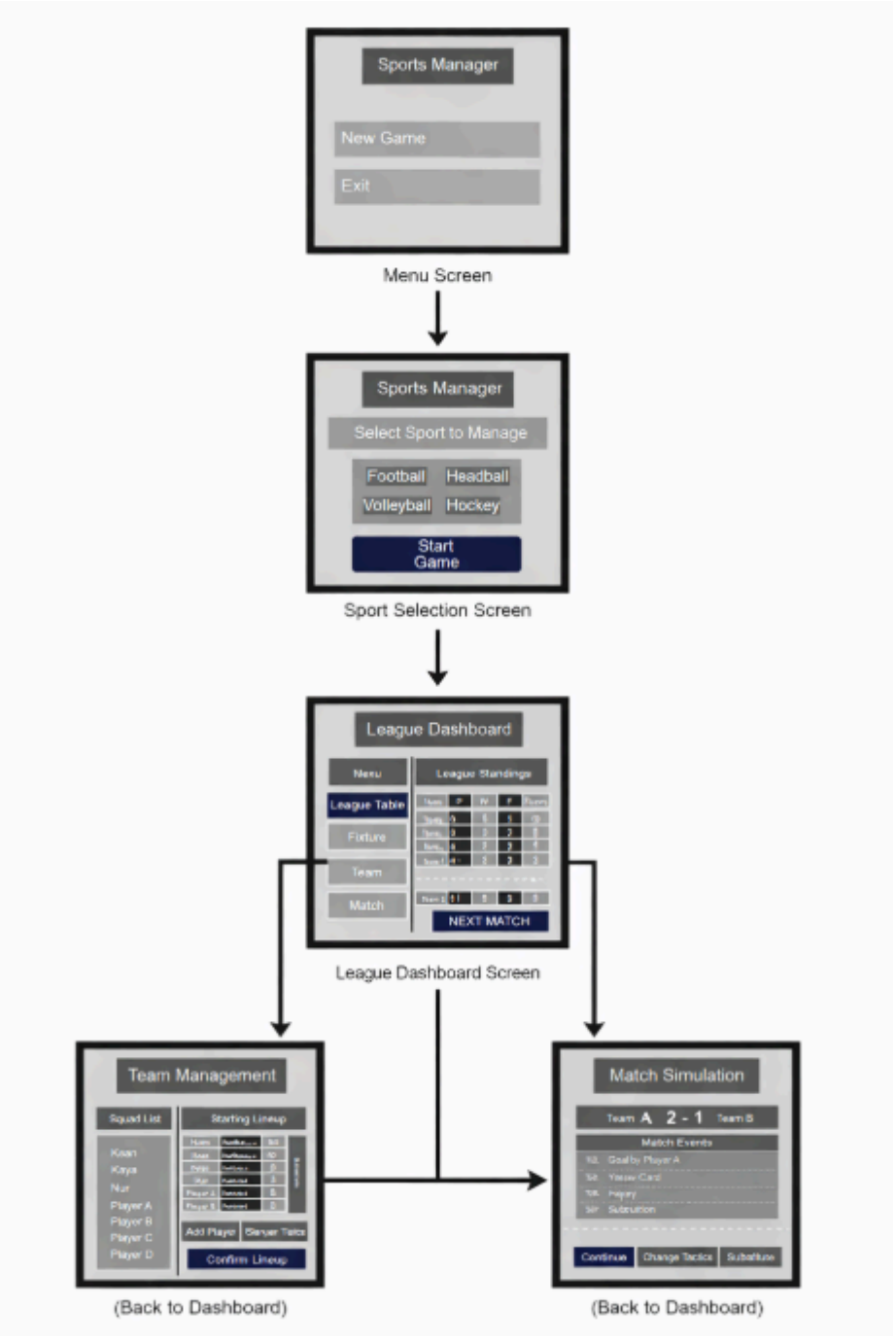

Figure 6.6 : Navigation Flow

7 Risks and Assumptions

7.1 Risks

- The match simulation logic may become complex when new sports are added to the system.
- If the separation between the user interface and sport-specific logic is not implemented correctly, it may be harder to extend the system in later stages of development.
- Mistakes in designing core entities such as teams, matches or fixtures may cause problems when trying to improve or modify the system in the future.

7.2 Assumptions

- The application is expected to run on a Windows environment with Java Runtime already installed.
- It is assumed that new sports can be added by following the defined sport interface without requiring major changes in the existing system structure.
- Player specifications and match results are assumed to be generated using simplified simulation logic instead of highly realistic one.

8 Conclusion

In this report, the general design and architectural approach of the Sports Manager application has been explained. The primary objective of the project is to develop a sports management simulation system with a clear and modular structure capable of supporting multiple sports disciplines. The system was designed using object-oriented design principles, such as abstraction and separation of concerns, so that sport-specific rules can be implemented independently of the user interface and core domain components. Requirement analysis and domain modeling helped to define the system's main entities and how they interact during a league-based competition. UML diagrams were created to visually represent the system's structure and guide the implementation phase. Also, potential risks and assumptions were identified to better understand the project's limitations and expectations. In general, suggested design provides a structured framework for the development of the Sports Manager application. This approach is expected to make the development process more organized and allow for future improvements without needing major changes to the existing structure.